School Van Tracking: Ordering Guarantees Under Unreliable Networks

Anand Theertha

Independent researcher

November 2026

Abstract

We study the telemetry protocol of a real-time school-van tracking application in which a driver's phone streams GPS positions to parents over unreliable mobile networks. Using TLA+ and the TLC model checker, we model *two* unordered channels (driver→server and server→parent) with loss on the uplink and unbounded redelivery of the same sequence number—receives do not remove frames from the channel sets. We check that the position shown to a parent never moves backward (`MonotonicDisplay`). TLC reports a violation in the unguarded baseline; a seven-state counterexample is documented with raw logs in the public artifact. An ablation study shows that, under this downlink model, *the parent display guard alone is sufficient* to restore monotonicity: a server-only guard still allows the map to jump backward when frames 1 and 2 both sit on the parent channel. Production deploys *both* guards so server state and ancillary consumers (e.g., trip history) stay consistent, not because the server guard alone secures the live map. We also report fault-injection tests on the implementation (duplicate, reordered, and batched frames). We state precisely what the model does and does not prove and outline follow-on work (session epoch, shared acknowledgment state, retransmission).

Index Terms—formal verification, TLA+, model checking, GPS telemetry, mobile networks, message ordering, transportation systems.

1 Introduction

Parents who track a school van need a map position that reflects the vehicle's recent path. If the displayed position jumps *backward* along the route—because an older GPS frame is applied after a newer one—they may infer the wrong direction, stop, or miss a pickup window. Cellular dead zones, reconnect bursts, and multi-node ingestion make such orderings plausible even when each TCP or WebSocket connection is in-order locally.

This paper is an applied case study: we (1) formalize a minimal telemetry protocol and channel assumptions in TLA+, (2) use TLC to find a monotonicity violation and to compare server-side versus client-side sequence guards, and (3) relate the model to a deployed Node.js/React Native stack through fault injection. Our contributions are:

- A two-channel TLA+ model (lossy uplink, unordered downlink, redelivery) that abstracts reconnect and multi-publisher reordering as a channel assumption, with reproducible TLC logs.
- A guard *ablation*: under the stated downlink, the parent display guard alone restores `MonotonicDisplay`; server-only guarding does not.
- A concise mapping from abstract violations to production mitigations and known remaining gaps (epoch on driver restart, cross-node races, liveness without acks).

Artifact: https://github.com/Anand-Theertha/school-van-formal-verification (release v1.2.0); archived copy DOI https://doi.org/10.5281/zenodo.23567441.

2 System overview

The driver app samples GPS every two seconds and sends each position as a frame with trip id, monotonic `seq_id`, timestamp, and coordinates:

1. Driver app (React Native) sends frames over Socket.IO on secure WebSockets.
2. One or more Node.js ingestion nodes validate the token, persist to PostgreSQL/PostGIS, update a current-trip cache, and publish via a Redis pub/sub adapter.
3. Parent apps subscribed to the trip receive the broadcast and render the position.

The update rate is bounded by the sampling interval; measured average end-to-end latency was about 250 ms on 4G in testing. Appendix A summarizes stack, schema, and security choices.

3 Baseline protocol and failure model

Baseline protocol. The server accepts each valid frame and broadcasts it. Without sequence guards, the parent display follows whatever arrives on the downlink.

Failure model.

- Connections drop and reconnect; buffered frames may arrive in bursts.
- Application-level retries can duplicate deliveries.
- Frames from an old socket can arrive after frames from a new connection.
- Several server nodes can process frames for the same trip concurrently.

Out of scope. GPS error, dead reckoning, map matching, and malicious clients.

4 Formal model (TLA+)

4.1 Channel abstraction

TCP and WebSockets do not reorder frames within one connection. We nevertheless use *unordered* channels because that is a conservative abstraction of overlapping old and new sockets during reconnect, application retries, and parallel processing on multiple ingestion nodes.

We use two channels:

- **Uplink** (`network_buffer`): driver→server; **lossy** (`NetworkDrop`).
- **Downlink** (`parent_channel`): server→parent; **unordered** in the model.

Deployment note. Redis pub/sub may preserve order from a *single* publisher. We model an unordered downlink because multiple ingestion nodes can publish toward the same trip, so parents need not see frames in send order. Multiple publishers are not modeled explicitly: there is a single server with one shared `server_ack_seq`, so downlink reordering is an assumption of the channel rather than a consequence of modeled nodes.

Duplication. Each channel is a set of sequence numbers; receives do *not* remove an element, so the same `seq_id` can be delivered arbitrarily many times (redelivery). We do not model two distinct in-flight copies of the same `seq_id` in one set (a multiset would be needed for that).

A counterexample in this model shows the protocol *abstract* can violate a property under these assumptions; it is not a proof about the live cellular network or the full production codebase.

4.2 State variables

Variable	Meaning
`driver_seq`	Latest sequence number produced by the driver
`network_buffer`	Set of sequence numbers in flight on the uplink
`server_ack_seq`	Highest sequence number the server has accepted
`parent_channel`	Set of sequence numbers forwarded toward the parent
`parent_view_seq`	Sequence number currently displayed

4.3 Actions

The specifications include exactly these actions (no separate "connection drop" or "duplicate" operators):

Action	Description
`DriverSend`	Driver produces the next frame (`driver_seq` increases, up to `MaxSeq`)
`NetworkDrop`	Uplink loss: remove one frame from `network_buffer`
`ServerReceive`	Server reads one frame from `network_buffer`; may append to `parent_channel`
`ParentReceive`	Parent reads one frame from `parent_channel`; updates `parent_view_seq`

Modules (ablation).

- `SchoolVanTracker_Baseline`: no guards.
- `SchoolVanTracker_ServerGuarded`: server guard only.
- `SchoolVanTracker_ParentGuarded`: parent guard only.
- `SchoolVanTracker_Guarded`: both guards, as deployed.

4.4 Bounds and specification

`MaxSeq` = 3 in all modules (finite state space for TLC). There is no bound on drops or redeliveries beyond finiteness of the model. `Spec == Init /\ [][Next]_vars` with **no fairness** hypothesis for the safety runs below. TLC configurations set `CHECK_DEADLOCK FALSE` (terminal states exist when the driver has finished and both channels are empty).

5 Properties

Safety (action property): the displayed position never moves backward.

```
MonotonicDisplay == [][parent_view_seq' >= parent_view_seq]_parent_view_seq
```

Safety (invariant): only positions the driver has produced are displayed.

```
NoFutureFrame == parent_view_seq <= driver_seq
```

Liveness (not checked in current TLC configs):

```
FinalDelivery == (driver_seq = MaxSeq) ~> (parent_view_seq = MaxSeq)
```

`FinalDelivery` requires stronger fairness, bounded loss, or retransmission. Section 9 discusses acknowledgments for a future model revision. The TLC runs in this release check `MonotonicDisplay` and `NoFutureFrame` only.

6 Model checking results

Environment. TLC 2.20 (`tla2tools.jar` via VS Code TLA+ extension), Temurin JDK 25, Windows 11, 2026-10-23. Raw logs ship in directory `tlc/` (Appendix B).

6.1 Baseline protocol

TLC **violates** `MonotonicDisplay`. Shortest counterexample: **7 states** (six steps: two `DriverSend`, two server receives, two parent receives).

State	Summary
1	Init
2–3	Driver sends seq 1 and 2; `network_buffer = {1,2}`
4–5	Server accepts 1 then 2; `parent_channel = {1,2}`
6	Parent displays 2 (`parent_view_seq = 2`)
7	Parent applies 1 → `parent_view_seq = 1` (**violation**)

Reordering on the *downlink* suffices; uplink duplication is not required for this trace. At error: 282 states generated, 122 distinct (search stopped with 64 states left on queue). Trace depth 7.

6.2 Guard ablation (`MonotonicDisplay`)

Variant	Module	Result	Generated	Distinct	Depth
No guards	Baseline	Violated	282	122†	7
Server only	ServerGuarded	Violated	251	97†	7
Parent only	ParentGuarded	Holds	1726	347	11
Both (v1)	Guarded	Holds	922	199	11

†First violation stops exploration early; distinct counts are not the full state graph.

Conclusion. Under an unordered `parent_channel`, the **parent guard alone is sufficient** for `MonotonicDisplay`; the server guard alone is not. The server-only failure follows directly from the channel assumption: the server forwards 1 then 2 in order, and the set-based `parent_channel` redelivers 1 after the parent has shown 2. Both guards remain appropriate in production for server/history consistency and stale-frame filtering on the uplink.

"Holds" means no violation within `MaxSeq = 3` and the stated channel model; it is not a proof for unbounded sequences or all real-network behaviors.

6.3 Liveness

`FinalDelivery` was not run in the bundled `.cfg` files. Informally, without driver-side retention and server acknowledgments, a lost final frame can leave the parent view stalled (aligned with injection test T6 in Section 8). That is an explicit v1 trade-off: safety properties were the focus of model checking.

7 Protocol revisions (production v1)

Server filter. Drop any frame whose sequence number is less than or equal to the server's current highest seen value for that trip (uplink guard).

Parent guard. Render a frame only if its sequence number is greater than the last one displayed.

TLC ablation (Section 6) shows the parent guard is sufficient for monotonic *display* under the modeled downlink; the server guard supports consistent server state and consumers other than the live map.

8 Implementation validation (fault injection)

Test	Injected fault	Expected after fix	Baseline	v1
T1	Old frame after newer	No backward jump	14 jumps	0 jumps
T2	Duplicate frame	No visible change	UI flickers	Ignored
T3	Burst after reconnect	Final position correct	Jumps	Correct
T4	Driver restart, counter resets	Frames accepted	Accepted	Dropped (stale)
T5	Two nodes concurrent	No backward jump	Jumps	Jumps (cross-node race)
T6	Last frame lost before reconnect	Final position shown	Stalls	Stalls

9 Limitations and future work

- **Session epoch** (T4): local `seq_id` reset without epoch can freeze the trip.
- **Idempotent history:** store all frames with (trip, epoch, sequence); separate history from live broadcast.
- **Shared server state** (T5): atomic Redis update for highest seen seq across nodes.
- **Acknowledgments and retransmission** (T6): needed for `FinalDelivery`-style liveness.
- **Model:** optional multiset for duplicate in-flight copies; explicit downlink loss; per-action fairness for liveness checks.

10 Related work

This work sits at the intersection of applied formal methods and distributed telemetry. Lamport's TLA+ and TLC toolchain [1] is the verification backbone. Kleppmann [2] discusses ordering, idempotency, and stream processing patterns relevant to guarded ingestion. Lamport's logical clocks [3] motivate sequence-based reasoning without claiming equivalence to our guards. Beschastnikh et al. [4] survey debugging distributed systems; our counterexample traces play a similar explanatory role at design time.

11 Conclusion

Unreliable networks and multi-node publish paths can expose parents to non-monotonic map updates even when each connection is locally ordered. A small TLA+ model with explicit uplink loss and an unordered downlink yields a reproducible seven-state violation and a guard ablation that clarifies *where* monotonicity must be enforced: at the parent display under our assumptions. Production adds a server guard for broader consistency; closing epoch, cross-node, and liveness gaps remains future work. Artifacts and TLC logs are public for independent replay.

Data and code availability

TLA+ modules, TLC configuration files, unedited console logs (`tlc/`), modeling notes (`MODELING.md`), and this preprint source are available at https://github.com/Anand-Theertha/school-van-formal-verification. A versioned archive is deposited at Zenodo (concept DOI, resolving to the latest version: https://doi.org/10.5281/zenodo.23567441).

References

[1] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.

[2] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.

[3] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[4] I. Beschastnikh, P. Wang, Y. Brun, and M. D. Ernst, "Debugging distributed systems: Challenges and options for validation and debugging," *Communications of the ACM*, vol. 59, no. 8, pp. 32–37, 2016.

A Architecture and schema

Stack. Node.js/Express; Socket.IO over WSS with Redis pub/sub; PostgreSQL 15 with PostGIS; React Native driver and parent apps.

Schema (conceptual). Users, routes, children, trips, and location samples keyed by trip with monotonic `seq_id`, geospatial columns (PostGIS), and time partitioning for retention.

Security. JWT (RS256); bcrypt password hashing; role-based publish/subscribe; TLS 1.3 on public endpoints.

Design decisions. WebSockets over polling for low latency; PostgreSQL/PostGIS over document stores for geospatial queries; shared ordering responsibility across driver buffering, server filtering, and parent display guards.

B TLC log excerpt (baseline violation)

The following lines are excerpted from `tlc/baseline-monotonic-2026-10-23.txt` (TLC 2.20, 2026-10-23). Full logs for baseline, server-guarded, parent-guarded, and both-guarded runs are in the repository.

Listing 1: Excerpt: error state (state 7).

```
7: <Next line 55, col 8 to line 55, col 49 of module SchoolVanTracker_Baseline>
/\ network_buffer = {1, 2}
/\ driver_seq = 2
/\ server_ack_seq = 2
/\ parent_view_seq = 1
/\ parent_channel = {1, 2}

282 states generated, 122 distinct states found, 64 states left on queue.
The depth of the complete state graph search is 7.
Finished in 8140ms at (2026-10-23 15:02:04)
```